HUM1012 : Logic and Language Structure

(LT)

Slot : B21 (Mon 04.25 pm – 05.55 pm) + E14 (Tue 02.50 pm – 04.20 pm)
D21 (Tue 01.15 pm – 02.45 pm) B14 + (Wed 02.50 pm – 04.20 pm)

Venue : AB02 – 301 (B21+E14)
AB02 – 301 (B14+D21)

Prepared and Presented by

Dr. Velmani Ramasamy, M.E., Ph.D., SMIEEE., MACM.,

Associate Professor (Grade 2)

School of SCAI,

VIT Bhopal University,

Bhopal, Madhya Pradesh.

MODULE 5 Typed Lambda Calculus (TLC) and Function Application in Natural Language 6

Type semantics, lambda abstraction, syntax and semantics of lambda abstraction, lambda calculus, and linguistic application, equality, negation, binary connectives, quantification, Beta-reduction.

Function application with grammatical categories (content and functional), determiners and quantifiers, conditions and quantifiers, and predicate modification (type semantics and type shifting)..

COURSE OUTCOMES:

CO5: Understanding and knowledge acquired form the course will enable students to take up further courses based on natural language processing in term of machine.

5.1 Typed Lambda Calculus

- Lambda calculus (λ-calculus), originally created by Alonzo Church by 1930-40's.
- It is the world's smallest programming language.
- The lambda calculus can be thought of as the theoretical foundation of functional programming.
- It is a Turing complete language.

- The Lambda calculus is an abstract mathematical theory of computation, involving 'λ' functions.
- The 'λ' notation is based on function abstraction and application based on variable binding and substitution.

Lambda calculus is composed of 3 elements:

- ✓ variables
- ✓ Functions.
- ✓ applications.

Name	Syntax	Example	Explanation
Variable	<name>	x	a variable named "x"
Function	λ<parameters>.<body>	λx.x	a function with parameter "x" and body "x"
Application	<function><variable or function>	(λx.x)a	calling the function "λx.x" with argument "a"

5.1.1 Type Semantics

- Assigns semantic types to linguistic expressions (e.g., noun phrases, verbs, sentences).
- Prevents ill-formed meaning combinations (e.g., applying a truth value directly to a noun).

Basic Types:

e → entities (individuals like *John*, *Mary*).

t → truth values (*True/False*).

Complex Types:

<e,t> → functions from entities to truth values
(e.g., predicates like is tall).

<e, <e,t>> → relations
(e.g., loves → takes two entities and returns True/False).

Functional types:

- If A and B are types, then (A → B) is also a type.

Example:

Predicate: happy : e → t (takes an entity, returns a truth value)

Determiner: every : (e → t) → ((e → t) → t)

Applications in NLP:

- Word meaning representation.
- Sentence-level semantics.
- Formal representation for reasoning in AI.

Tools:

- Type-driven semantic parsers.
- Montague Grammar frameworks.
- λ-calculus libraries in Python (e.g., nltk.sem).

5.1.2 Lambda Abstraction

- Function creation using the λ-operator.
- A way to create functions in semantics.

Syntax: λx. M

(a function that takes x and returns expression M).

Type: If M : B when x : A, then λx. M : (A → B).

Example:

λx. tall(x) → function that maps an entity *x* to True if *x* is tall.

"runs" → λx. run(x) (runs : e → t)

Applications:

Representing generalized functions (predicates, quantifiers).

Handling variable binding in natural language.

Tools:

NLTK semantic module (nltk.sem.logic).

Coq / Isabelle theorem provers for type-checked λ-expressions.

5.1.3 Syntax and Semantics of Lambda Abstraction

- Combine words into phrases compositionally.

Syntax:

λ-expression → λvariable. expression

Application → (λx. P(x)) (a)

Variables: x, y, z Lambda terms: x | (M N) | (λx. M)

Semantics:

λ-abstraction defines a function.

Application substitutes the argument for the bound variable.

E.g:

John runs => λx. run(x))(John) → run(John)

John is tall => Abstracting: "is tall" = λx. tall(x)

λx. tall(x)(John) → tall(John)

Example:

John loves mary

$(\lambda x.\ loves(John, x))(Mary) \rightarrow loves(John, Mary)$

Applications:

Resolving meaning composition in syntax-semantics interface.

Modeling scope ambiguity (Every student read a book).

Tools:

Haskell (functional programming syntax mirrors λ-calculus).

Prolog with λ-extensions.

5.1.4 Lambda Calculus (General)

- A formal system for defining functions and applying them.
- Acts as a bridge between syntactic structure and semantic interpretation.

Core operations:

Abstraction ($\lambda x.E$) → defines a function.

Application (F A) → applies function 'F' to argument 'A'.

Example

"John runs"

John : e

runs : e → t

runs(John) : t

"Every student runs"

student : e

runs : e → t

every : (e → t) → ((e → t) → t)

Full form: every(student)(runs) → ∀x (student(x) → runs(x))

Applications in NLP:

Compositional semantics.

Quantifier handling.

Logical form generation.

Tools:

Montague Grammar implementations.

Python + NLTK for semantic parsing.

5.1.5 Linguistic Application

- Natural language is compositional.
- The meaning of a sentence depends on its parts and structure.
- Words are assigned semantic types.
- Phrases and sentences are formed via function application (just like in lambda calculus).
- Sentences reduce to truth values (t).
- TLC allows us to:

  Represent meanings as typed functions.

  Ensure only valid combinations (via types).

  Compute sentence meaning step-by-step (beta-reduction).

- Representing verbs: $\lambda x.\ runs(x)$
- Representing determiners: $\lambda P.\ \lambda Q.\ \exists x\ (P(x) \wedge Q(x))$ (*a student*).
- Handling ambiguity: scope of quantifiers (*Every student loves some book*).

Applications:

Semantic parsing.

Question answering.

Natural language inference.

Tools:

OpenAI semantic parsers.

NLTK's DrtParser for Discourse Representation Theory.

5.1.6 Equality

- Expressing equivalence of meanings.
- Equality expresses identity of denotation.

Example:

John = Jack (true if they are the same entity).

John = the teacher.

Type: e → e → t.

Clark Kent is Superman" → =(Clark Kent, Superman).

Applications:

Coreference resolution.

Ontology alignment in NLP.

Tools:

First-order logic solvers.

Description Logic reasoners (e.g., Protégé).

5.1.7 Negation

- Logical operator flipping truth value: ¬P.
- Negation flips truth values.
- ¬P (if P : t, then ¬P : t).
- Function: ¬ : t → t.

Example in λ-calculus:

λP. λx. ¬(P(x)) → function that negates predicate.

"John does not run" → ¬run(John).

Applications:

Sentiment analysis (*not*).

Representing negative statements in logic.

Tools:

NLTK logic module.

Prover9 for first-order logic.

5.1.8 Binary Connectives

- Logical operators combining propositions.
- Logical connectives combine truth values. i.e., **∧** (and), **∨** (or), **→** (implies).

  Conjunction (∧ : t → t → t) → "and"

  Disjunction (∨ : t → t → t) → "or"

  Implication (→ : t → t → t)

Example:

λP. λQ. λx. (P(x) ∧ Q(x))

"John runs and Mary sings" → run(John) ∧ sing(Mary).

Applications:

Representing compound sentences.

Inference in natural language.

Tools:

Automated theorem provers.

Logic frameworks in Python.

5.1.9 Quantification

- Handles expressions like "every", "some", "no".

Types:

Universal Quantifier (∀):

Universal ($\forall x\ P(x)$) → All x satisfy P.

$\forall : (e \rightarrow t) \rightarrow t$

Example: All student runs => $\forall x\ (student(x) \rightarrow runs(x))$

Existential Quantifier (∃):

Existential ($\exists x\ P(x)$) → Some x satisfies P.

$\exists : (e \rightarrow t) \rightarrow t$

Example: Some Student runs => $\exists x\ (student(x) \wedge runs(x))$

Example in λ-calculus:

every student → $\lambda P.\ \forall x\ (student(x) \rightarrow P(x))$

a student → $\lambda P.\ \exists x\ (student(x) \wedge P(x))$

"Some student runs" → $\exists x\ (student(x) \wedge run(x))$.

Applications:

Natural language understanding.

Question answering.

Representing scope ambiguities.

Tools:

NLTK quantifier-handling.

Formal semantic parsers.

5.1.10 Beta-reduction

- Applying functions to arguments by substitution.
- The computation rule in lambda calculus: applying a function to an argument.
- Rule: $(\lambda x.\ M)\ N \rightarrow M[x := N]$
- Step where sentence meaning is computed.

Example:

$(\lambda x.\ loves(John, x))(Mary) \rightarrow loves(John, Mary)$

$(\lambda x.\ run(x))(John) \rightarrow run(John)$

$(\lambda P.\ \forall x\ (student(x) \rightarrow P(x)))(run) \rightarrow \forall x\ (student(x) \rightarrow run(x))$

Applications:

Semantic composition in NLP.

Simplification of logical forms.

Tools:

NLTK's λ-calculus engine.

Functional programming languages (Haskell, OCaml).

NLP Example

Sentence: **"Every student runs"**

Lexical entries:

student : $e \rightarrow t$

runs : $e \rightarrow t$

every : $(e \rightarrow t) \rightarrow ((e \rightarrow t) \rightarrow t)$

$= \lambda P.\ \lambda Q.\ \forall x\ (P(x) \rightarrow Q(x))$

Lambda application:

Every(student)(runs)

$= (\lambda P.\ \lambda Q.\ \forall x\ (P(x) \rightarrow Q(x)))(\text{student})(\text{runs})$

Beta-reduction:

$(\lambda Q.\ \forall x\ (\text{student}(x) \rightarrow Q(x)))(\text{runs})$

$\rightarrow \forall x\ (\text{student}(x) \rightarrow \text{runs}(x))$

Final meaning: **"For all x, if x is a student, then x runs."**

Concept	Type/Meaning in TLC	NLP Example
Type Semantics	e, t, (A → B)	runs : e→t
Lambda Abstraction	λx. run(x) → function from entities to truth values	"runs"
Syntax/Semantics	(λx. tall(x))(John) → function application	"John is tall"
Lambda Calculus (General)	Formal system with variables, abstraction (λx.M), application (M N)	Used to compute meaning compositionally: (λx. run(x))(John) → run(John)
Linguistic Application	Maps natural language syntax to logical semantics via typed functions	Sentence "Every student runs" → ∀x (student(x) → run(x))
Equality	= : e → e → t (identity between entities)	"Clark is Superman" → Clark = Superman
Negation	¬ : t → t (truth value inversion)	"John does not run" → ¬run(John)
Binary Connectives	∧, ∨, → : t → t → t (combine truth values)	"John runs and Mary sings" → run(John) ∧ sing(Mary)
Quantification	∀ : (e→t)→t, ∃ : (e→t)→t	"Every student runs" → ∀x (student(x) → run(x))
Beta-reduction	Function application rule (λx. M) N → M[x:=N]	(λx. run(x))(John) → run(John)

Concept	Type/Meaning	Application in NLP	Tools
Type Semantics	e, t, <e,t> typing system	Word/sentence semantics	NLTK, Montague Grammar
Lambda Abstraction	λx.E function definition	Predicate/quantifier modeling	NLTK, Coq
Syntax & Semantics	Abstraction + Application rules	Composition of sentence meaning	Haskell, Prolog
Lambda Calculus	Function abstraction + application	Logical form generation	NLTK
Linguistic Application	Meaning of verbs, quantifiers	Semantic parsing, inference	DRT parsers
Equality	x = y	Coreference resolution	Protégé, FOL solvers
Negation	¬P	Negated meanings, sentiment	NLTK logic
Binary Connectives	P ∧ Q, P ∨ Q, P → Q	Compound sentences	Theorem provers
Quantification	∀, ∃	Universal/existential meaning	NLTK
Beta-reduction	Substitution during application	Sentence-level meaning composition	NLTK, Haskell

5.2 Functional Applications

- Core operation in formal semantics.
- Type rule:

  If α = type ⟨a,b⟩ and β = type *a*, then α(β) = type *b*.

Applications

- Sentence meaning construction in Montague grammar.
- Natural language processing (NLP): parsing → semantics.
- Inference in AI: mapping sentences to logical forms.
- Programming languages: function–argument evaluation (lambda calculus basis).

Tools

- Lambda calculus (formalism).
- Combinatory Categorial Grammar (CCG) and Montague Grammar.
- NLP toolkits (e.g., NLTK, spaCy semantic parsing modules).

5.2.1 Function Application in Semantics

- In Montague-style semantics, meaning is treated as functions applied to arguments.
- A sentence is built compositionally: each syntactic category has a semantic type.
- Function application (FA):

  If an expression of type ⟨a, b⟩ (a function from type *a* to type *b*) combines with an expression of type *a*, the result has type *b*.

Example:

- "Speaks Mary" = λx. speaks(x, Mary)

  (function from individuals *e* → truth values *t*).
- "John" = an individual (*type e*).
- Applying FA:

  (λx. speaks(x, Mary))(John) = speaks(John, Mary).

  → Type t (a truth value).

5.2.2 Grammatical Categories

- We distinguish content words (open-class) vs functional words (closed-class).

Content words (nouns, verbs, adjectives, adverbs):

Denote predicates, individuals, or relations.

Semantic types:

Noun: ⟨e,t⟩ (set of individuals)

Verb (transitive): ⟨e, ⟨e,t⟩⟩

Adjective: ⟨e,t⟩ modifier

Functional words (determiners, quantifiers, complementizers, etc.):

Operate on content words.

Example:

Determiner *every* maps a predicate (noun) and a predicate (VP) into a truth value.

Applications

- Linguistic categorization → automatic tagging (POS tagging).
- Syntax–semantics interface (assigning meaning compositionally).
- Language teaching (function vs content words in acquisition).
- Computational models (transformers use function–content separation implicitly).

Tools

- Treebanks (Penn Treebank, Universal Dependencies).
- POS taggers in NLP.
- Compositional semantics frameworks.

5.2.3 Determiners and Quantifiers

- Determinants like *every, some, the, no* are functions over sets.
- Determiners: generalized quantifiers (⟨⟨e, t⟩, ⟨⟨e, t⟩, t⟩⟩).
- Quantifiers: logical operators ∀, ∃, ∃!, etc.

Example:

"Every student studying"

- student: λx. student(x) (type ⟨e,t⟩)
- studying: λx. study(x) (type ⟨e,t⟩)
- every: λP.λQ. ∀x [P(x) → Q(x)]
- FA: every(student)(study) = ∀x [student(x) → study(x)]
- So determiners shift simple predicates into quantificational expressions.

Applications

- Information retrieval (queries like *all documents containing X*).
- Question answering (who/what/which → quantified expressions).
- Ontology-based reasoning (e.g., *every mammal is an animal*).
- Knowledge representation in AI (first-order logic).

Tools

- Description Logics (OWL, RDF, SPARQL) for knowledge bases.
- Theorem provers / SAT solvers (handling ∀, ∃).
- Semantic parsing models (mapping natural language to FOL).

5.2.4 Conditions and Quantifiers

- Quantifiers introduce conditions on sets (scope, restriction, nuclear scope).

  Universal (∀) → condition on all individuals in a set.

  Existential (∃) → condition on at least one individual.

Restrictor + nuclear scope separation.

Restrictor: the set defined by the NP (e.g., *students*).

Scope (nuclear scope): the VP predicate (e.g., *laugh*).

Example:

"Some student passed"

- Restrictor: {x | student(x)}
- Scope: {x | passed(x)}
- Meaning: ∃x [student(x) ∧ passed(x)]
- Conditions: Universal (∀), existential (∃), uniqueness (∃!), etc.

Applications

- Database query languages (SQL uses ∃, ∀ implicitly).
- Logic programming (Prolog).
- Formal verification (checking conditions under constraints).
- Natural language entailment (if *all A are B* and *C is an A*, then *C is B*).

Tools

- First-order logic systems.
- Constraint solvers (SMT, Z3).
- Database systems (SQL quantifiers).
- NLP entailment models (e.g., SNLI, NLI systems).

5.2.5 Predicate Modification (Type Semantics & Type Shifting)

Predicate Modification

- When two predicates combine to form a single predicate.
- ⟨e,t⟩ + ⟨e,t⟩ → ⟨e,t⟩ (intersection).

Rule:

If both are type ⟨e,t⟩, then conjunction is applied pointwise.

Example:

- adjective "tall" = λx. tall(x)
- noun "student" = λx. student(x)
- "tall student" = λx. [tall(x) ∧ student(x)]
- This is predicate modification, combining two ⟨e,t⟩ predicates.

Type Shifting

- Sometimes categories don't match type expectations.
- Type shifting rules fix mismatches.
- Type shifting rules to adjust mismatched types (e.g., individuals → generalized quantifiers).

Lift individuals to predicates:

- John (type e) → λP. P(John) (type ⟨⟨e,t⟩,t⟩).
  → lets John be treated as a generalized quantifier.

Shift predicates to quantifiers:

- Student (⟨e,t⟩) → λQ. ∃x[student(x) ∧ Q(x)].
- This allows flexible composition.

Applications

- Compositional semantics in adjective–noun phrases (*tall student*).
- Type coercion in NLP (*the book is heavy* vs *the book is interesting* → physical vs informational object).
- Word sense disambiguation.
- Ontology alignment (shifting between individuals and sets).

Tools

- Lambda calculus rules for type shifting.
- Type-logical grammars.
- Formal ontology frameworks (e.g., SUMO, WordNet).
- Computational Semantics libraries (like pyccg, MontaguePy).

Function Application in Natural Language

Concept	Semantic Type	Example	Rule/Meaning
Function Application	⟨a,b⟩ + a → b	speaks(Mary)(John)	Apply function to argument
Content words	e, ⟨e,t⟩, ⟨e,⟨e,t⟩⟩	nouns, verbs	Basic denotations
Functional words	higher-order types	determiners, quantifiers	Operate on predicates
Determiner	⟨⟨e,t⟩,⟨⟨e,t⟩,t⟩⟩	every, some	Restrictor + scope
Quantifier condition	∀, ∃	every student studied	logical condition
Predicate Modification	⟨e,t⟩ + ⟨e,t⟩ → ⟨e,t⟩	tall student	conjunction
Type Shifting	e → ⟨⟨e,t⟩,t⟩	John → λP.P(John)	resolves mismatch

Concept	Type	Applications	Tools
Function Application	⟨a,b⟩ + a → b	Sentence meaning, NLP parsing	Lambda calculus, CCG, NLTK
Grammatical Categories	Content: ⟨e,t⟩, Functional: higher-order	POS tagging, syntax–semantics mapping	Treebanks, POS taggers
Determiners & Quantifiers	⟨⟨e,t⟩,⟨⟨e,t⟩,t⟩⟩	IR, QA, ontology reasoning	OWL, RDF, theorem provers
Conditions & Quantifiers	∀, ∃ conditions on sets	Databases, Prolog, entailment	SQL, SMT solvers, NLI models
Predicate Modification & Type Shifting	⟨e,t⟩ + ⟨e,t⟩ → ⟨e,t⟩; type coercion	WSD, ontology alignment	Lambda rules, type-logical grammars

Thank You!